The Hong Kong Polytechnic University

Department of Computing

COMP4913 Capstone Project Report (Interim)

Enterprise Data Leakage Prevention System

Student Name:	Kong Wing Sum Timothy
Student ID No.:	22077237D
Programme-Stream Code:	46409-INS
Supervisor:	Dr Xie Hongcheng
Co-Examiner:	Dr TAYYAB Muhammad
2nd Assessor:	Dr Leong Hong-va
Submission Date:	08 January 2026

Abstract

Our world is slowly turning digital, with our data even being handled digitally. Although this is making our lives easier and more convenient, a new threat of the leakage of our data and privacy also has risen.

This shows the major challenge that is slowly emerging in our world: Data Leakage. It is not a simple problem, but one that even large technology companies such as Google and Apple have faced in the past. Once data leakage happens, the consequences may be unbearable. Personal information or company confidential information may be stolen, and misused for illegal purposes, under our identity.

In counter of this major challenge, solutions do exist. For my solution, it is about creating a secure enterprise level file sharing system, with techniques such as secure file encryption, password hashing, Audit Logging of all events in the system, Watermark embedding in files shared, viewed and edited on the system, as well as an AI Model to help identify suspicious activities and flag sensitive information, all to help trace action events and find out sources of possible danger to information stored in the system.

Table of Contents

ABSTRACT **2**

TABLE OF FIGURES **4**

1. INTRODUCTION **5**

2. PROBLEM FORMULATION **7**

2.1 SYSTEM OVERVIEW 7
2.2 THREAT MODEL 9
2.3 PRELIMINARIES 10
2.4 DESIGN GOALS 12
1. Data Analytics Engine *12*
2. Bare minimum Admin Privileges *13*
3. File Self-Destruction *13*
4. Watermark Processing *13*

3. RELATED WORKS **14**

4. SYSTEM DESIGN **15**

4.1 THEORETICAL DESIGN 15
4.1.1 Database *15*
4.1.2 Frontend *17*
4.1.2.1 Login and Logout Logic *17*
4.1.2.2 Register Logic *19*
4.1.2.3 File Uploading Logic *20*
4.1.2.4 File Viewing/Editing Logic *21*
4.1.2.5 File Deleting Logic *22*
4.1.3 Backend *23*

5. DESIGN SHOWN THROUGH CODE (IMPLEMENTATION) **25**

5.1 Index.html *25*
5.2 Register *26*
5.3 Login *30*
5.4 Upload File *32*
5.5 Logging *34*

6. SECURITY ANALYSIS **37**

7. ELEMENTS OF THE PROJECT TO BE COMPLETED **37**

8. CONCLUSION **39**

REFERENCES **40**

Table of figures

Figure 1 A Flowchart of the Data Leakage Prevention System 8
Figure 2 Entity Diagram of the System 9
Figure 3 An example of Shift rows 11
Figure 4 User Table 15
Figure 5 File Table 16
Figure 6 AuditLog Table 16
Figure 7 Login Flowchart 18
Figure 8 Logout Flow Chart 18
Figure 9 Register Flow chart 19
Figure 10 Upload File Logic 20
Figure 11 File Viewing Logic 21
Figure 12 File Editing Logic 22
Figure 13 File Deleting Logic 23
Figure 14 Index Page 25
Figure 15 Code for handling Register events 27
Figure 16 Code of User Model 28
Figure 17 Code of handling Register event: Adding new user and new Auditlog 29
Figure 18 Exception Handling 30
Figure 19 Login Function 31
Figure 20 Upload File Function: File checking 32
Figure 21 File Uploading Function: File Checking and Logging for failed upload 33
Figure 22 File Uploading Function: Adding file to model and database + Logging event 33
Figure 23 Logging Function 34
Figure 24 Logging function used on login event 34
Figure 25 AuditLog Table viewed on localhost/phpMyAdmin 35
Figure 26 Audit logs viewed from the user dashboard 36
Figure 27 An example of an Audit log with status = 'failed' 36

1. Introduction

Technology has been always evolving in our world, and up till this date, the level of technology is higher than ever. The rise in the development of AI and computing applications has forced people to follow suit, incorporating technology and computing in their daily lives, such as communicating through social media or paying others with credit cards. As a result, much information is put on social media and the internet, which also includes many sensitive information and data such as our name, residential address, or even credit card information. Although this information is said to be "safely and securely stored", has that always been the case?

There have been major data leaks in the past, with examples being Yahoo in 2013, resulting in the exposure of over 3 billion user accounts [1]; Facebook in 2018, with them exposing sensitive data of 533 million users to Cambridge Analytica [2]; as well as PayPal in 2022, with them being the victim of a credential stuffing incident that led to the leakage of 35,000 Social Security Numbers [3]. In these 3 examples, the victims are all large companies, that take care of many users' personal and sensitive information.

Data Leakage is a very serious issue, because once data is leaked to the outer world, people get to know your sensitive information and can do what they want with it. Confidential information and documents may be leaked, which could result in problems such as Identity theft and financial loss for an individual, or financial loss, legal liability and reputation damage for an organization, as mentioned by the HKSAR Digital Policy Office [4], examples including exposing residential details of an individual, or even leakage of confidential documents regarding non-disclosure agreements. But when Mark Zuckerburg, the CEO of Facebook, said that he would "ban developers that had misused personally identifiable information", and "tell everyone affected by those apps", is it really a good strategy to resolve data leakage problems only after they get leaked? Will it be too late to do so? When even big companies are prone to data leakage threats, is the digital world that we are living in truly a safe place?

Preventing data leakage is of crucial importance for everyone and each organization, to ensure the protection of everyone's data and identity, yet is something that is often overlooked. In my opinion however, with the rise in the development of AI, the issue of data leakage must be

highlighted as soon as possible and resolved before it gets too late. For me, my work would try to solve these challenges by only having a secure data sharing platform. In there we only allow the minimum needed for admins to work, and incorporating steganography into documents by adding visible watermarks whenever a user shares or edits a file, so that the documents wouldn't be leaked by outside people such as admins, or even at the worst possible case the documents get leaked, we would be able to trace back the timeline and find the point of leakage.

My contribution to the work would be trying to incorporate a different perspective of working with digital data, which is by continuously tracking user activity and to require everything to be done on the platform, providing extra layers of security while at the same time allowing employees and other people in the company to perform their work.

2. Problem Formulation

Data Leakage is the main problem that will be tried to solve. With the continuous rise in developments of Technology and Artificial Intelligence (AI), more information from our everyday life will be stored as digital data, and we must take care of them very carefully to prevent accidents that could lead to large consequences.

For the Capstone Project, the final goal is to create a Data Leakage Prevention System (DLPS) that prevents the leakage of a company's data and especially sensitive data, to the outer world. The system will be crafted in a web application form, and is done by setting rules in the application on how data is transmitted within the platform, to ensure data is:

- being managed correctly,
- unable to be leaked through the user's actions on the computer, and:
- encrypted to protect it from attacks.

2.1 System Overview

The Data Leakage Prevention System is primarily used for the sharing, viewing and modifying of files, and we would make sure they are done in a secured manner. Users of the system can share one file to one user at a time, and can edit a file and share it to one user at a time as well. The system will be used by employees and managers of an organization. Admins are pre-set and it should not be possible to add a new admin.
There will be a user module, a file module, a sharing module, and an AuditLog module in the system. An Audit Log will be added upon every major event happened in the system, such as login attempts, register attempts, and file uploading/editing/sharing attempts. Users can view logs associated with their own account, and admin can view logs associated with every single user.

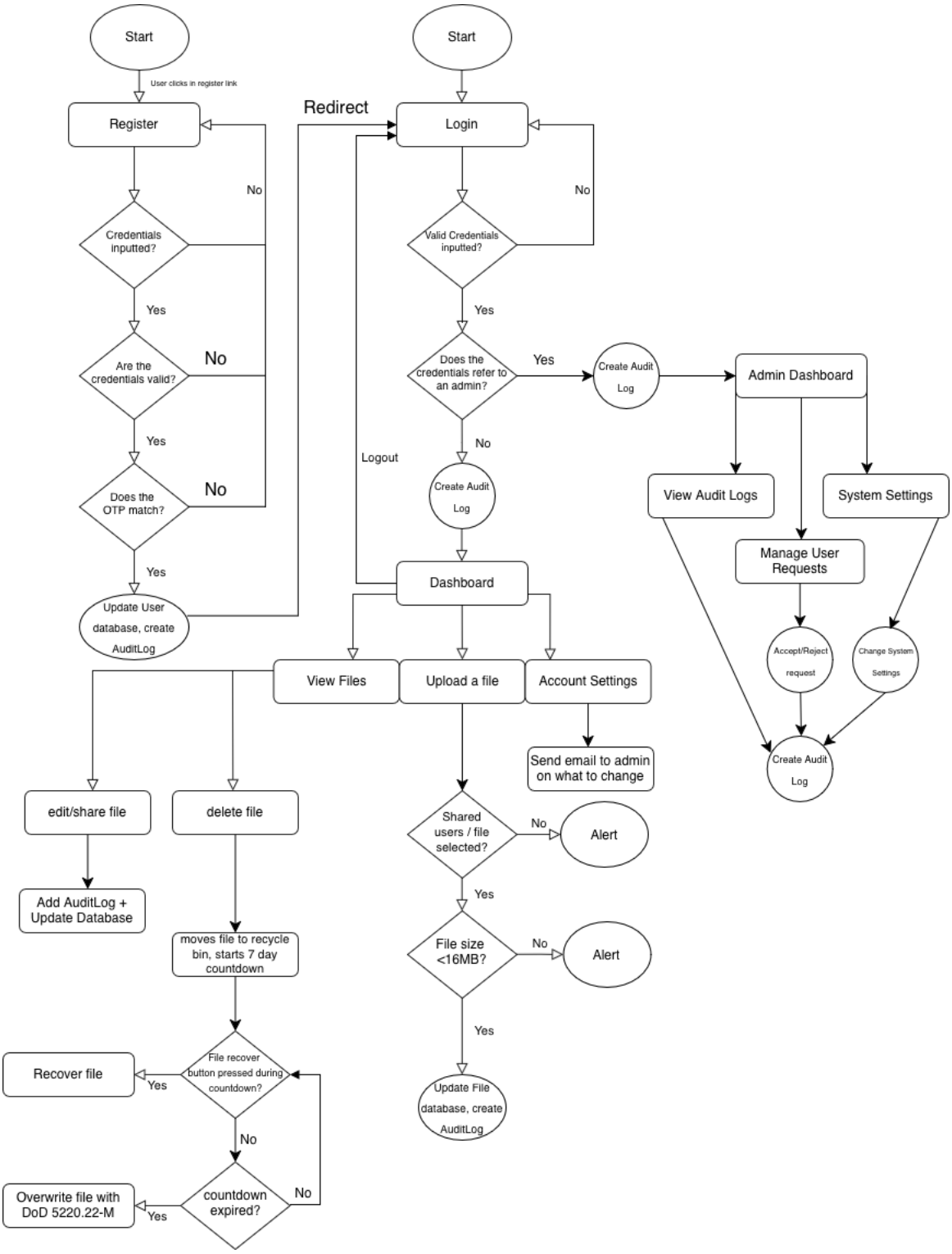


Figure 1 A Flowchart of the Data Leakage Prevention System

2.2 Threat Model

In the project, we need to create a system that prevents data leakage of documents and data. The elements to be protected are: User's personal information, and the contents of files shared and uploaded to the system. The files could range from digital-related information such as design files and source code to intellectual-related information such as trade secrets and certifications. Data integrity is also a goal to be achieved through this project.

The entities of the system only includes users and the web server.

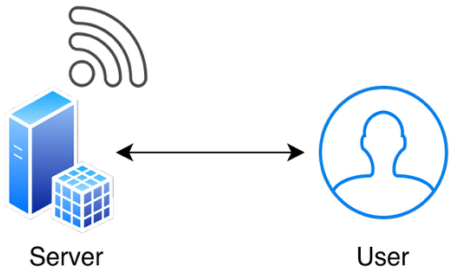


Figure 2 Entity Diagram of the System

In this system, we would consider the users of this system to be semi-trusted, in the sense that the user will follow the workflow but may sometimes be interested with the web page or with the admin privileges, or the users may leak documents that should be kept secret.

Other possible threats to this system from the users may include:

- Admin Leaking documents, or User information
- Users trying to attack the website by a Denial-of-service (DoS) Attack
- Cross-Site Scripting
- Storage Flooding/Database overload
- Steganography: Hidden malware/illegal data in images/audio files
- Exposure of data through screenshots of the data on the system, or hardware capture
- Attacking the system through weak encryption points: SSL/TLS Vulnerabilities, Forcing HTTP instead of HTTPS

We will aim to prevent these hacks in this project. Although screenshots and hardware capture of data cannot be fully prevented, we can try our best to trace back the source of screenshot or capture and take legitimate action on preventing the user from doing the same in the future.

For this project however, it is assumed that Python's in-built cryptographic functions and open-source libraries are secure, and the AI Model to be used for Risk scoring is also secure.

2.3 Preliminaries

AI Model ------ An AI Model will be used in this system for the risk scoring part and data analytics engine, where the system has to raise a sensitivity score for a file just uploaded to the system via uploadfile.html, which indicates the risk of sharing the file and/or uploading it to the system. Microsoft Azure Information Protection, an existing model will be used for this step, as this model can help classify files by their sensitivity, and does text scanning along with pattern matching to do so [5].

The AI model will be used when a user uploads a file, and then the AI model will raise a risk score, which is a number ranging from 1-10 that marks the level of sensitivity. The higher the number, the more confidential the document is. For example, a .txt file containing some names of restaurants would result in a lower number like 2, while a .txt file containing some trade secrets or credit card numbers would result in a higher number such as 8 or 9.

The AI model will also be used for determining if an action regarding the viewing, editing or sharing of a file performed is suspicious, and will flag suspicious activities and the flag will be included in the AuditLogs.

Scrypt ------ Scrpyt is a hashing method in-built in Python's Cryptographic Library and is used in the in-built generate_password_hash() function, which takes in the password as an argument and adds a 16-bit long salt for storing the users' passwords in a hashed ciphertext form, to prevent exposure of the plaintext passwords.

AES-128 Encryption ------ an Advanced Encryption Standard (AES) that encrypts and decrypts data in blocks of 128 bits is an encryption method and is typically used when uploading a file to the system, as well as when the user views and edits the file on the system.

In AES, each block is considered as a 4 byte x 4 byte grid, arranged by column. The encryption step of AES contains 4 steps: SubBytes, ShiftRows, MixColumns and Add Round Key. This round of 4 steps is performed 10 times. Therefore, there would contain 10 round keys, 1 for each subsequent round[6].

Sub Bytes is about substituting each byte with another byte, according to a table called the S-box. It is performed in a way that the original byte will never be substituted with the same byte.

Shift Rows is just about shifting each row by a certain number of times. The first row is not shifted, the 2nd row is shifted once to the left, the 3rd is shifted twice to the left and the 4th row will be shifted once to the right.

```
[ b0  | b1  | b2  | b3  ]        [ b0  | b1  | b2  | b3  ]
| b4  | b5  | b6  | b7  |   ->  | b5  | b6  | b7  | b4  |
| b8  | b9  | b10 | b11 |        | b10 | b11 | b8  | b9  |
[ b12 | b13 | b14 | b15 ]        [ b15 | b12 | b13 | b14 ]
```

Figure 3 An example of Shift rows

Mix Columns consists of multiplication of matrices, and the multiplication of each column with a matrix will result in the bytes of each column to be changed.

Add Round Keys is about XORing the output from the previous 3 stages altogether with a round key according the the round of encryption. We first perform the 1st round of 4 steps with round key 1, then the 2nd round with round key 2, and so on.

For Decryption, the steps are similar to encryption but the steps are reversed. The process of MixColumns and SubBytes alre also reversed, by using a different matrix and a different S-box respectively to perform the steps.

DoD 5220.22-M ------ Regarding file destruction, a data overwriting technique of **DoD 5220.22-M** will be used**.** It is about overwriting the previously stored data with 0s and 1s 3 times. The first time will be overwriting the addressable location of the file with binary 0s, then the second time will be overwriting the addressable location of the file with binary 1s, and the third time

will be overwriting the addressable location of the file with a random bit pattern [7]. This is to ensure that the file will never be possible to recover.

Pillow — A Python Library used for embedding watermarks into different formats of images [8].

PyMuPDF — A Python Library used for merging text or images into each page of a document [9].

DeepSeek (GenAI) is also used as a guideline for the steps on implementing certain functions, such as uploading files process.

2.4 Design Goals

To make my application stand out and be different from other DLPs, the web application needs to store data securely, allow some people to monitor it from time to time, be able to create digital evidence as preparation for once accidents happen, and be able to detect sensitive data by itself. To do so, the project will be created with these 4 major features:

1. Data Analytics Engine

1.1 Pattern Recognition

This feature is to help the application identify abnormal or suspicious activities according to the time of accessing the file, time of data export for a certain file, or time of transferring a file.

1.2 Risk Scoring

This feature helps provide the user a reference about the level of risk for a data transfer.

1.3 Data Classification & User Reminder

This is to help the application identify files with sensitive data, exclusively label them and prevents the user from sharing a file containing confidential or sensitive information carelessly. Tools such as Microsoft Azure Information Protection and techniques such as machine learning, as well as clustering may be used in the making process.

2. Bare minimum Admin Privileges

This is about incorporating admins into the platform and allowing them to take care of user registration and logging analysis, as well as monitoring the application to ensure that it runs smoothly. Admins should not be able to access any documents uploaded to the system, to prevent data leakage of any document information from overpowered admin privileges.

3. File Self-Destruction

This feature is to completely overwrite past files/past data so that they can never be recovered and thus leaked.

This feature works by having a “Recycle bin” to store files deleted by a user temporarily, with the bin allowing the user to restore the files before having the system to completely overwrite the data after the finish of a 7-day countdown. The Recycle bin also has an option for the user to immediately delete a file, which will bypass the 7-day countdown and immediately overwrite the file data with an overwriting method named DoD 5220.22-M.

4. Watermark Processing

This feature is to enable tracing of actions done on a specific file, so that if a suspicious action is done on a certain file, the time of suspicious action done and the employee responsible for the action can be known.

This is done by using the steganography technique with the use of libraries like Pillow or PyMuPDF, with embedding watermarks or words into the file, and tracing the embedding history of that file once suspicious activity is detected.

3. Related Works

An example of a failed DLP would be Apple in 2022 [10], where there was found to be a bug in the Safari app, that allowed information such as browsing history and Apple IDs to be leaked, allowing malicious websites to use this exploit to see URLs that a user recently visited and thus knowing their Google User ID, which can allow cybercriminals to find personal user info. The reason for DLP failure would be them not completely following the least privilege principle, where people are able to access data that they were not supposed to.

The disadvantage of not having least privilege could be hazardous. Consequences could range from outsiders knowing our information to identity theft on doing illegal actions, putting others at risk.

For my application however, I will always ensure the principle of least privilege is being properly used and everyone only gets to have access to the bare minimum they needed to use the application.

Another example of failed DLP would be Yahoo in 2014, where they experienced a data breach about some hackers from Russia somehow was able to retrieve the entire user database of Yahoo, and Yahoo did not even notice the breaching incident until 2 year later in 2016 [11]. This is due to the insufficient monitoring of Audit logging, leading to this incident. If they have realized that a hacker has been retrieving their information at the time, the incident may have been less of a catastrophe.

Therefore, for my application I will take Audit logging very seriously, even logging failed logins due to wrong password on the user dashboard, and logging the modifications of the SQL database to let admins see it, so that they can realize if an SQL intruding attack is ongoing.

4. System Design

4.1 Theoretical Design

The idea of the project is to allow users to upload and share files on this intranet platform and allow users to specify who gets access to the file they uploaded. All the files uploaded to this platform are encrypted with encryption algorithms and passwords are all hashed to provide extra layers of security. Logging is incorporated in all parts of the platform, and logs will be updated when the user first registers, logs in, logs out, uploads (and shares) a file and edits a file. Admins would be able to monitor all users' activities through logging and are responsible for user management.

4.1.1 Database

The database dlps.sql contains tables for storing: User entries, File entries, and AuditLog entries. Sharing entries and editing entries will be introduced and implemented in the future.

For the User table, we store a newly created id and take in the fullname, username, email, password and position from the registration form and store them into the SQL Database.

```
CREATE TABLE User (
    ID INT PRIMARY KEY AUTO_INCREMENT,
    fullname VARCHAR(255) NOT NULL,
    username VARCHAR(255) NOT NULL,
    email VARCHAR(50) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL,
    position VARCHAR(50) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    is_admin BOOLEAN DEFAULT FALSE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Figure 4 User Table

The is_admin variable is default set to false as users are not allowed to create admin accounts.

For the File Table, we take in more parameters: file_id upon creation of the entry (Primary Key), the associated user's id (Foreign Key), the newly stored filename (designed to be unique), the original filename, filetype, file size, the content of the file, description of the file, shared with (To be modified and to be implemented), upload time (time of upload), status (To be implemented, used with Data Analytics Engine), sensitivity (To be implemented, used with Data Analytics Engine), and action (To be implemented, used with Data Analytics Engine).

```
CREATE TABLE File (
    file_id INT PRIMARY KEY AUTO_INCREMENT,
    user_id INT NOT NULL,
    filename VARCHAR(255) NOT NULL,
    original_filename VARCHAR(255) NOT NULL,  -- Original filename
    filetype VARCHAR(50) NOT NULL,  -- File type
    file_size BIGINT NOT NULL,    -- Original file size in bytes
    file_data LONGBLOB NOT NULL,  -- Actual file data
    description TEXT,      -- 32-byte random
    shared_with TEXT,   -- 32-byte random
    upload_time DATETIME DEFAULT CURRENT_TIMESTAMP,
    status VARCHAR(50) DEFAULT 'pending', -- pending, scanned, flagged, safe
    sensitivity INT,  -- 1-10
    action VARCHAR(50),  -- e.g., 'block', 'quarantine', 'alert'
    FOREIGN KEY (user_id) REFERENCES User(ID) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Figure 5 File Table

For the AuditLog table, we take in the user_id (Foreign Key) that the log is associated to, the type of action to be logged, some additional details about the log, timestamp of the event to be logged, the status (success or failed), and we assign a log_id to it.

```
CREATE TABLE AuditLog (
    log_id INT PRIMARY KEY AUTO_INCREMENT,
    user_id INT,
    action_type VARCHAR(50) NOT NULL,  -- e.g., 'upload', 'delete', 'share'
    details TEXT,  -- Additional details about the action
    status VARCHAR(50) DEFAULT 'success', -- success, failed
    timestamp DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES User(ID) ON DELETE SET NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Figure 6 AuditLog Table

4.1.2 Frontend

The frontend is crafted with basic HTML and CSS, along with the use of some JavaScript and jinja2 syntax. External framework Bootstrap is also used for the CSS of the frontend. The frontend includes:

- The index page (index.html)
- A Login page (login.html)
- A Registration page (register.html)
- Home page (dashboard.html), which includes:
    - A page for Checking new files sent (to me) (files.html)
    - A page for uploading and sharing files (uploadfile.html)
- Admin Home page (admin_dashboard.html), which includes:
    - A page for checking all logs of all users (audit_logs.html)
    - A page for managing users (manage_users.html)

4.1.2.1 Login and Logout Logic

When a user logs in, first, the web will check if a session variable exists. If it doesn't, the user has to input login credentials (i.e. username and password). The login credentials and password hash will be checked, and if it passes the check, the login request will be sent to the backend and the user will be given a session variable, added logs, and the user will be redirected to the dashboard page.

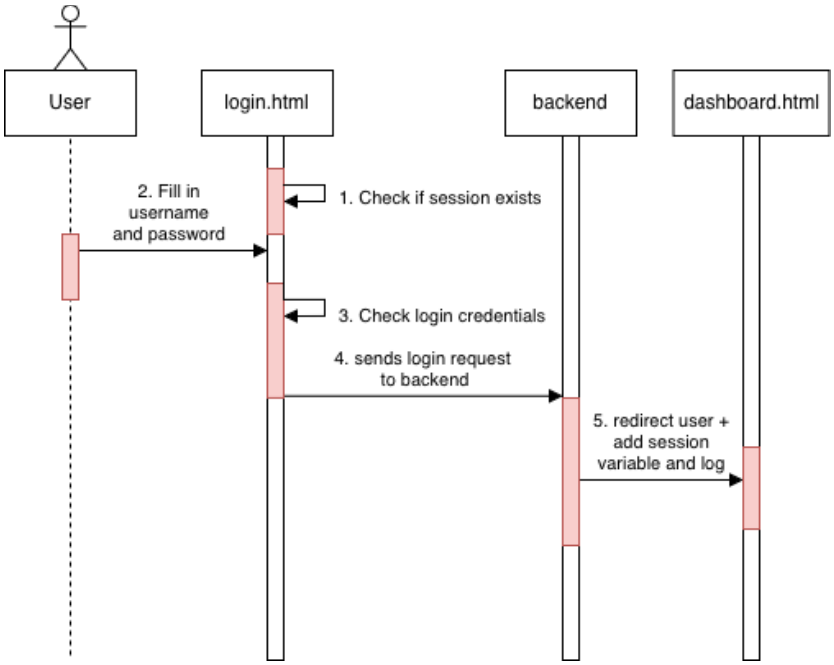


Figure 7 Login Flowchart

When a user logs out from the webpage, a logout() function request will be sent to the backend. Then the session variable of the user will be cleared, and the user will be redirected to index.html, without a session variable.

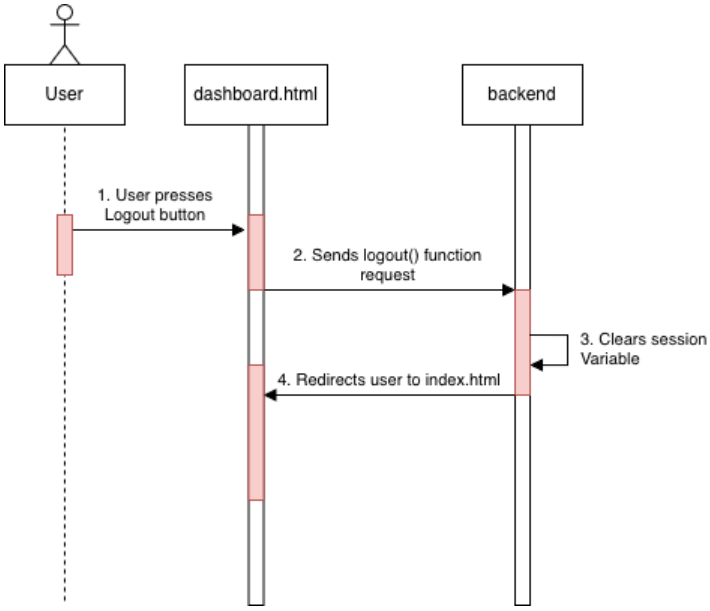


Figure 8 Logout Flow Chart

4.1.2.2 Register Logic

For the registration phase, The user will register through an email sent from the admin, which contains a one-time register link to the register page. The user then fills in credentials and clicks the “Sign Up” Button. Afterwards, the webpage first checks if all credentials are filled in and are filled in correctly, then the database will be fetched to see if both the username and email is new. If any of the above checks return False, the web will prompt the user to fill in the credentials again. Once the checks return True, the new user entry will be added and the registration event will be logged, and the user will be redirected to the login page.

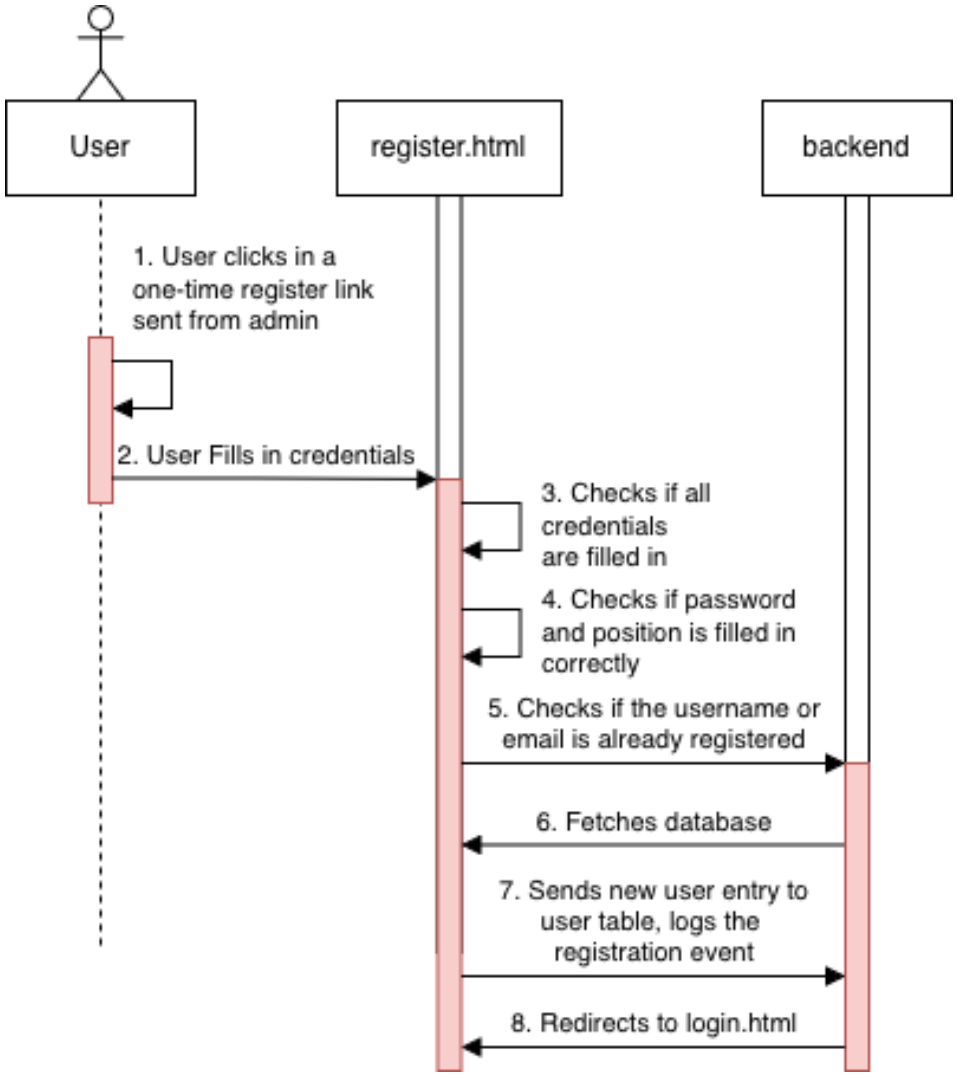


Figure 9 Register Flow chart

4.1.2.3 File Uploading Logic

When a user uploads a file, he/she has to select the user to share to first. Then, a small description can be added, and after the user selects the file to be uploaded, the system checks if the shared user and the file to be shared is selected when the user clicks the “Upload file” button. If any of the checkings return false, an alert will be raised and the user has to select the user or file accordingly. Else, the file will be uploaded to the system, with AES-128 being used to encrypt the file content. The shared user will also be able to see the new file being shared to him/her by the user that uploaded the file.

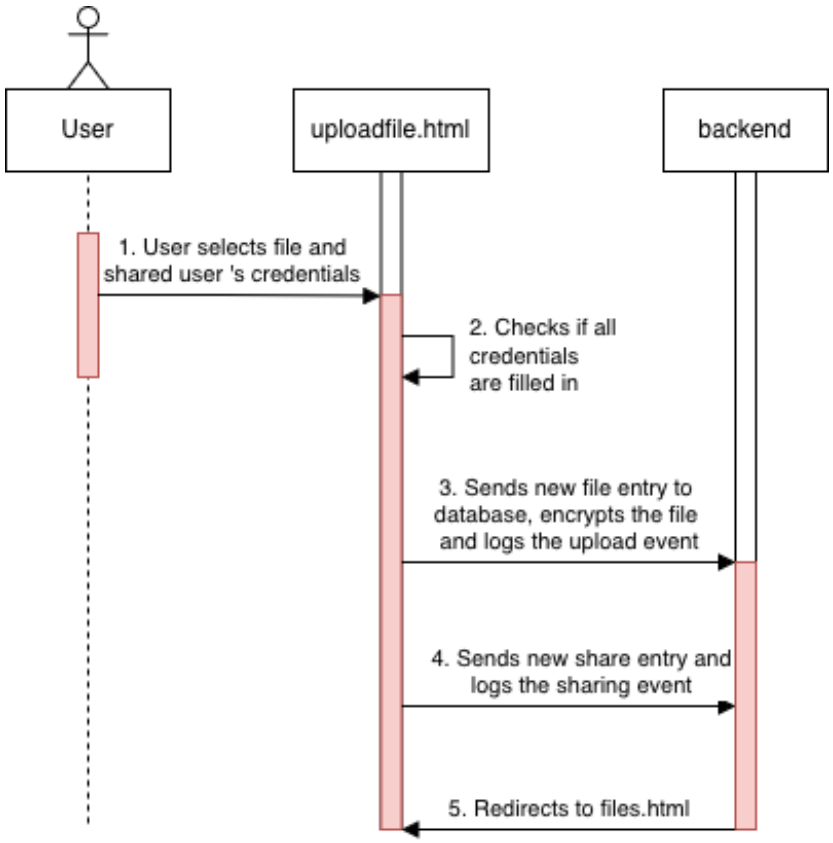


Figure 10 Upload File Logic

4.1.2.4 File Viewing/Editing Logic

When a user tries to view or edit a file, upon a “view” or “edit” button, the system will first fetch the corresponding file and its encrypted content, then decrypt the file using AES-128. Afterwards, the file would be able to be edited and viewed directly on the system.

Upon every view and edit of the file, the system will log an event based on the action of the user on that file. Also, a watermark will be embedded into the document selected, to track user’s actions and so that people can trace back the culprit when data leakage is associated with that file.

At last, the edited or viewed files can be chosen to share with other people on the system afterwards with the sharing function as well.

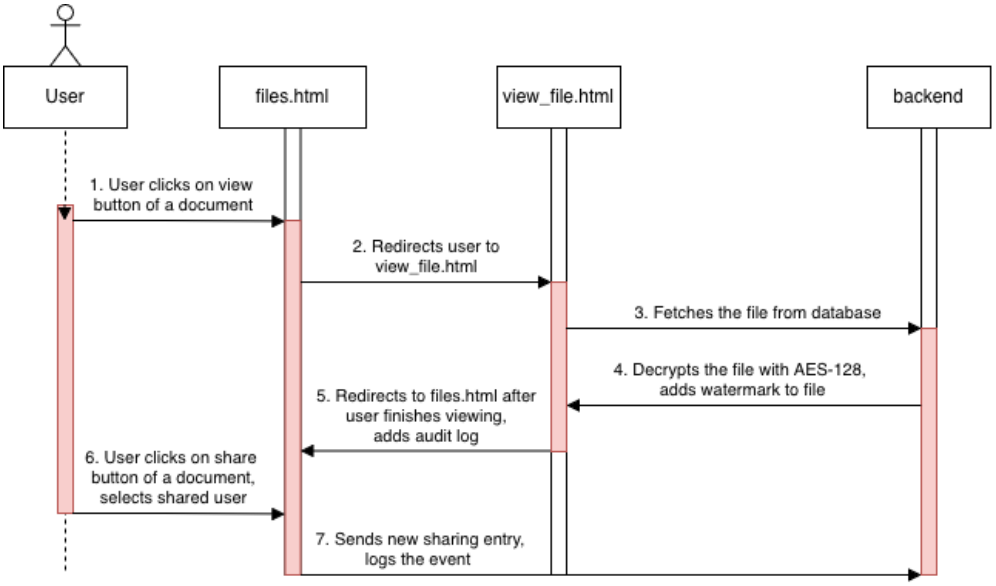


Figure 11 File Viewing Logic

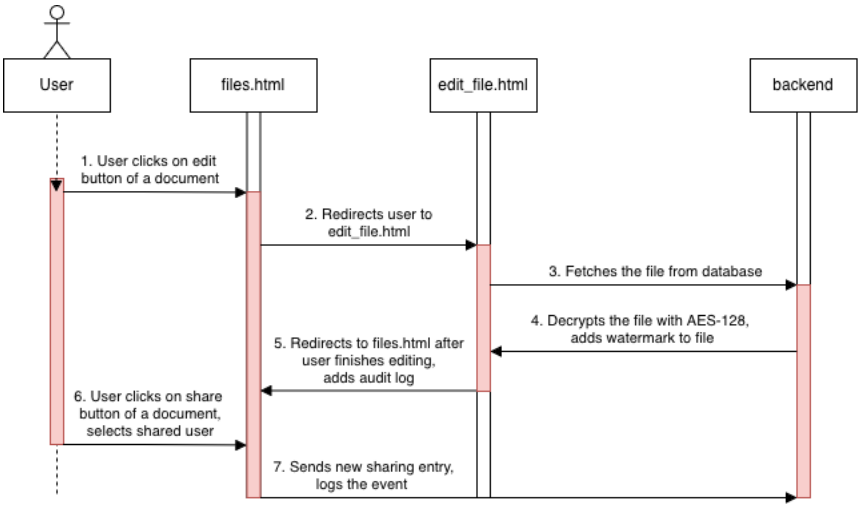


Figure 12 File Editing Logic

4.1.2.5 File Deleting Logic

When a user tries to delete a file, which is done by clicking on the “delete” button next to the corresponding file, the file first moves to a recycle bin and starts a 7-day countdown.

The file can be recovered from the recycle bin within the 7 days and the file will remain unchanged, but if the file remains in the recycle bin when the 7-day countdown expires, the file will be overwritten with DoD 5220.22-M method and be completely unrecoverable.

The file can always be deleted immediately and skipping the 7-day countdown upon a “empty recycle bin” request by the user.

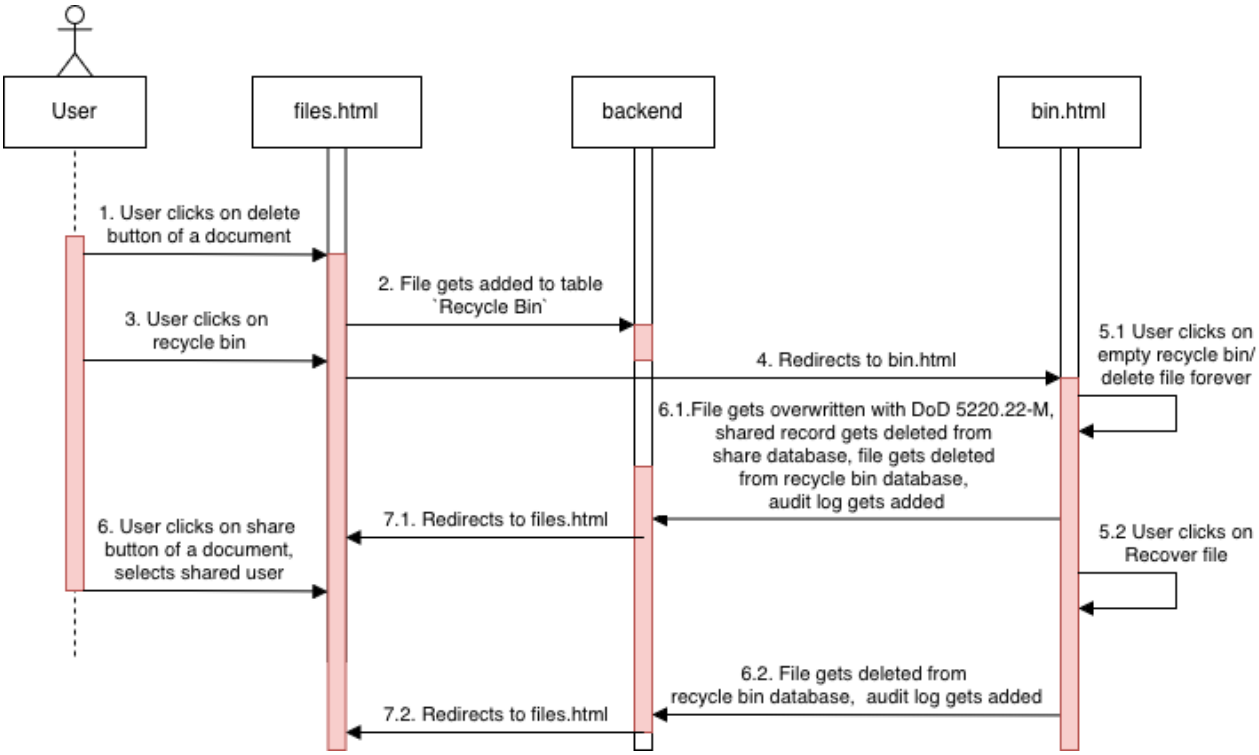


Figure 13 File Deleting Logic

4.1.3 Backend

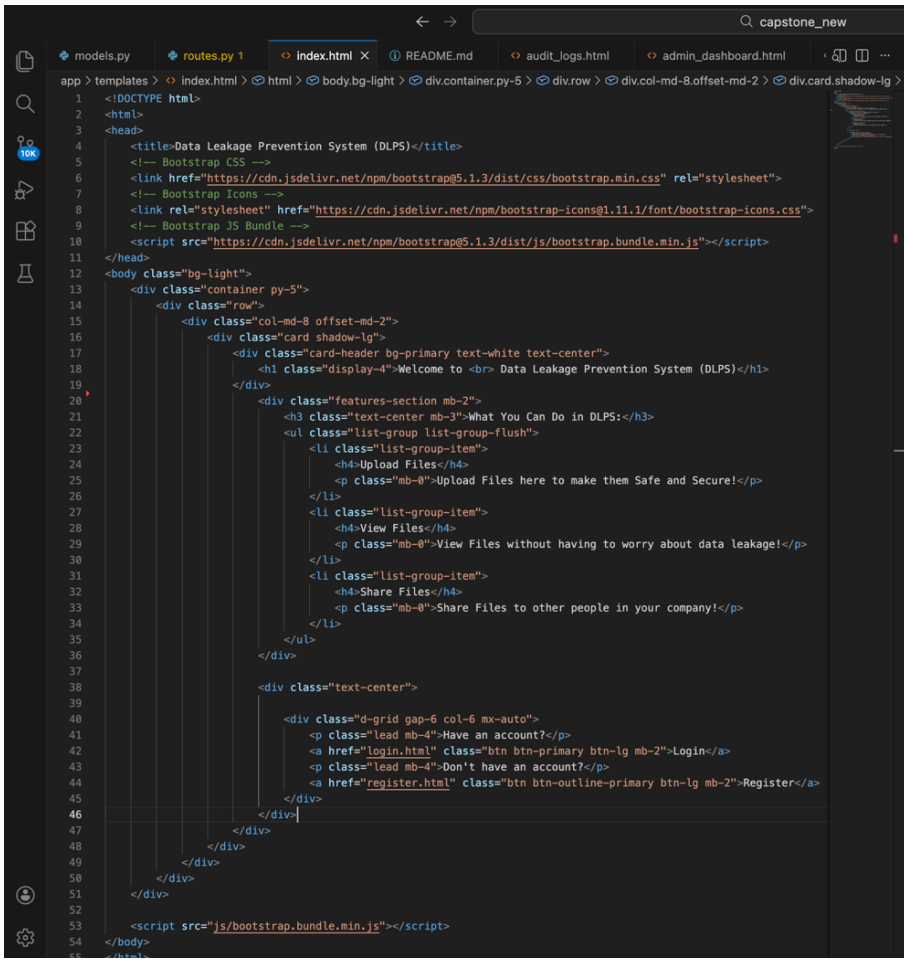
The backend is crafted with Python Flask, since it includes many cryptographic libraries for use. The following techniques is incorporated into the backend part of the application:

- Hashing: This would be used to store the members' passwords for their accounts securely in a database. Hashing algorithms such as SHA-256 will be used to create the hash values.
- Steganography: The technique of embedding things into another file would be included for the embedding of Watermarks, to fulfill the Watermark Processing feature of the project.
- One-Time-Pad (OTP): This would be used in the registration stage of each member when using the application, to verify the user.

- Encryption: Files uploaded to the application would be encrypted with methods such as AES-128, to protect them from being leaked and prevent attackers from being able to extract useful information from the files.
- Overwriting techniques: This is used in the file self-destruction, with algorithms such as random write or Zero dill being used, if logical destruction is used.
- Machine Learning: This technique may be included for the creation of the Data Analytics Engine for Pattern recognition to classify files' level of confidentiality and risk scoring of each file transfer.

5. Design shown through Code (implementation)

5.1 Index.html



```html
<!DOCTYPE html>
<html>
<head>
    <title>Data Leakage Prevention System (DLPS)</title>
    <!-- Bootstrap CSS -->
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.1.3/dist/css/bootstrap.min.css" rel="stylesheet">
    <!-- Bootstrap Icons -->
    <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.1/font/bootstrap-icons.css">
    <!-- Bootstrap JS Bundle -->
    <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.1.3/dist/js/bootstrap.bundle.min.js"></script>
</head>
<body class="bg-light">
    <div class="container py-5">
        <div class="row">
            <div class="col-md-8 offset-md-2">
                <div class="card shadow-lg">
                    <div class="card-header bg-primary text-white text-center">
                        <h1 class="display-4">Welcome to <br> Data Leakage Prevention System (DLPS)</h1>
                    </div>
                        <div class="features-section mb-2">
                            <h3 class="text-center mb-3">What You Can Do in DLPS:</h3>
                            <ul class="list-group list-group-flush">
                                <li class="list-group-item">
                                    <h4>Upload Files</h4>
                                    <p class="mb-0">Upload Files here to make them Safe and Secure!</p>
                                </li>
                                <li class="list-group-item">
                                    <h4>View Files</h4>
                                    <p class="mb-0">View Files without having to worry about data leakage!</p>
                                </li>
                                <li class="list-group-item">
                                    <h4>Share Files</h4>
                                    <p class="mb-0">Share Files to other people in your company!</p>
                                </li>
                            </ul>
                        </div>

                        <div class="text-center">

                            <div class="d-grid gap-6 col-6 mx-auto">
                                <p class="lead mb-4">Have an account?</p>
                                <a href="login.html" class="btn btn-primary btn-lg mb-2">Login</a>
                                <p class="lead mb-4">Don't have an account?</p>
                                <a href="register.html" class="btn btn-outline-primary btn-lg mb-2">Register</a>
                            </div>
                        </div>
                    </div>
                </div>
            </div>
        </div>
    </div>

    <script src="js/bootstrap.bundle.min.js"></script>
</body>
```

Figure 14 Index Page

The page index.html is just used as a welcoming page for people who just visited the website, showing what the website is for.

It also provides a link to login and a link to register.

5.2 Register

The frontend of the register is done with register.html, which includes a form with POST method, prompting the user to input their full name, username, email and verify it with an OTP, password, and their Position in the company, which is either a manager or an employee. Users cannot register as an admin due to security issues.

(The OTP function is yet to be implemented.)

Once the “Sign Up” button is clicked, a POST request is sent. Some checkings will be gone through:

1. Whether the Passwords match in the “Password” and “Confirm Password” box
2. Is the username new
3. Is the email address new
4. Is the Position filled in, and if so is it filled in correctly

If any of these checkings returns FALSE, relative Validation Errors will be raised, prompting the user to register again.

```python
@auth.route('/register', methods=['GET', 'POST'])
def register():
    try:
        if request.method == 'POST':
            fullname = request.form.get('fullname')
            username = request.form.get('username')
            email = request.form.get('email')
            password = request.form.get('password')
            confirm_password = request.form.get('confirm_password')
            position = request.form.get('position')
            #otp = request.form.get('otp')

            # Checking and confirm password match
            if password != confirm_password:
                raise ValidationError('Passwords do not match.')

            # Checking if full name is filled in
            if fullname is None or fullname.strip() == '':
                raise ValidationError('Please fill in your full name.')

            # Checking if position is filled in correctly
            if position is None or position.strip() == '':
                raise ValidationError('Please fill in your position.')

            if position not in ['Employee', 'Manager']:
                raise ValidationError('Invalid position. Please choose a valid position.')

            # Checking if the username is already taken
            if User.query.filter_by(username=username).first():
                raise ValidationError('Username is already taken. Please choose a different one.')
                #return render_template('register.html', username=username, email=email)

            # Checking if the email is already registered
            if User.query.filter_by(email=email).first():
                raise ValidationError('Email is already registered. Please choose a different one.')
                #return render_template('register.html', username=username, email=email)
```

Figure 15 Code for handling Register events

If all the checkings are passed and no Validation errors are raised, then the User gets added into the database and becomes an entry of the User Model class.

```python
models.py > ...
from flask_sqlalchemy import SQLAlchemy
from datetime import datetime
from werkzeug.security import generate_password_hash, check_password_hash
# from flask_wtf import FlaskForm
# from wtforms import StringField, PasswordField, SubmitField, SelectField
# from wtforms.validators import InputRequired, ValidationError, Length

db = SQLAlchemy()

class User(db.Model):
    __tablename__ = 'User'

    id = db.Column('ID', db.Integer, primary_key=True)
    fullname = db.Column(db.String(150), nullable=False)
    username = db.Column(db.String(80), unique=True, nullable=False)
    email = db.Column(db.String(80), unique=True, nullable=False)
    password = db.Column('password', db.String(255), nullable=False)
    #password_hash = db.Column(db.String(200))
    position = db.Column(db.String(50), nullable=False)  # Employee, Manager, Admin
    created_at = db.Column(db.DateTime, default=datetime.utcnow)
    is_admin = db.Column(db.Boolean, default=False)

    def set_password(self, password):
        self.password = generate_password_hash(password)

    def check_password(self, password):
        return check_password_hash(self.password, password)
```

Figure 16 Code of User Model

The User class contains the same information as in the User table in the phpMyAdmin database.

It also contains a set_password function, which hashes the plaintext password with the Werkzeug's (built in) generate_password_hash() function, and the check_password function is used in the login phase to verify if the hashed password inputted while login match the corresponding hashed password found in the database.

```
app > routes.py > register
173  def register():
209             # Creating a new user
210             new_user = User(
211                 fullname=fullname,
212                 username=username,
213                 email=email,
214                 position=position,
215                 is_admin=False
216             )
217             new_user.set_password(password)
218
219             db.session.add(new_user)
220             db.session.flush()  # Get new_user.id before commit
221
222             user_id = new_user.id
223
224             # Logging the registration event
225             log = auditlog(
226                 user_id=user_id,
227                 action_type='register',
228                 details='New user registration with email verification'
229             )
230             db.session.add(log)
231             db.session.commit()
232
233             flash('Account created successfully! You can now login.', 'success')
234             return redirect(url_for('auth.login'))
235
236         return render_template('login.html')
```

Figure 17 Code of handling Register event: Adding new user and new Auditlog

A new user is then created with the information inputted into the form as the parameters of the User class. The password is then hashed, and then we would add the new user information into the `User` Table in the database.

We then retrieve the new_user.id by db.session(flush), to help us log the registration event with the corresponding information: User ID, action (register in this case), and a small detail.

Then we add the auditlog to the `AuditLog` table in the database and commit the changes. Once everything is done successfully, the user gets redirected to the login page. If some other things are going wrong, an exception will be raised and the user will get redirected to the register page.

```
app > routes.py > register
def register():
                action_type='register',
                details='New user registration with email veri
            )
            db.session.add(log)
            db.session.commit()

            flash('Account created successfully! You can now l
            return redirect(url_for('auth.login'))

        return render_template('login.html')

    except Exception as e:
        print(f"ERROR in register: {type(e).__name__}: {e}")
        flash(f'Registration failed: {str(e)}', 'danger')
        return render_template('register.html',
                               username=username or '',
                               email=email or '')
    except SQLAlchemyError as e:
        db.session.rollback()
        print(f"Database error: {e}")
        flash(f'Database error: {str(e)[:100]}', 'danger')
        return render_template('register.html')
```

Figure 18 Exception Handling

5.3 Login

The login page is done with the help of a form with the use of POST method.

If the form is submitted, a POST request will be sent. We then first retrieve the inputted username and password and verify them from information stored in the database.

Once the inputted information is correct, we add a session variable to the user_id and username so that the user can log in and logging can be performed.
When a user is logged out, typing in the dashboard link on the web browser would lead to the login page instead of the dashboard itself.

After the session variables are given, we log the login event, and if the login credentials correspond to an admin account, the user will be redirected to admin_dashboard.html. For a regular user, they will be redirected to dashboard.html instead.

```
@auth.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        username = request.form.get('username')
        password = request.form.get('password')
        #remember = request.form.get('remember')

        if not username:
            flash('Please enter your username', 'danger')
            return render_template('login.html', username=username)
        if not password:
            flash('Please enter your password', 'danger')
            return render_template('login.html', username=username)

        user = User.query.filter_by(username=username).first()
        if not user:
            flash('Invalid username or password', 'danger')
            return render_template('login.html', username=username)

        if user and user.check_password(password):
            session['user_id'] = user.id
            session['username'] = user.username
            session['is_admin'] = user.is_admin
            flash('Login successful!', 'success')

            # Logging the registration event
            log = auditlog(
                user_id=session['user_id'],
                action_type='login',
                details='Successful login attempt'
            )
            db.session.add(log)
            db.session.commit()

            if user.is_admin:
                return redirect('/admin_dashboard')  # Or your admin route
            else:
                return redirect(url_for('main.index'))  # Regular user dashboard
        else:
            flash('Invalid username or password', 'danger')
            return render_template('login.html', username=username)


    return render_template('login.html')
```

Figure 19 Login Function

5.4 Upload File

Uploading of a file is done by a POST method. If the server receives a POST request, we first retrieve the filename, description and the shared_with users. Along with that, we will retrieve the file extension and see if its in the allowed extensions list. If it isn't in the list, we will not allow the upload and will prompt the user to upload a different file with a valid extension.

```
@main.route('/upload', methods=['GET', 'POST'])
def upload():
    try:
        if request.method == 'POST':
            file = request.files['file']
            if file.filename == '':
                flash('No selected file. Please choose a file to upload.', 'danger')
                return render_template('uploadfile.html')
            description = request.form.get('description')
            shared_with = request.form.get('shared_with')  # Comma-separated user IDs

            original_filename = file.filename

            def get_file_extension(filename):
                """Extract file extension including the dot"""
                if '.' in original_filename:
                    return '.' + filename.rsplit('.', 1)[1].lower()
                return ''

            filetype = get_file_extension(original_filename)

            if filetype not in ALLOWED_FILE_EXTENSIONS:
                flash('File type not allowed. Please upload a valid file.', 'danger')
                return render_template('uploadfile.html', shared_with=shared_with, description=description)
```

Figure 20 Upload File Function: File checking

The sharing function will not be implemented this way at last, but shared_with is just for temporarily making the upload function work for now.

Then we will create a unique filename for the uploaded file, as some file names may have same names at first and this may create confusions for the users and the database. Afterwards, we check if the file size is too large. If the file size exceeds a limit (16MB in this case), the file will not be uploaded, the user will be redirected to the login page, and an auditlog for upload failure will be logged.

```python
# Create unique filename
timestamp = datetime.now().strftime('%Y%m%d_%H%M%S')
user_id = session.get('user_id')
unique_filename = f"{user_id}_{timestamp}_{original_filename}"

#Read file data
file_data = file.read()
file_size = len(file_data)

if file_size > 16 * 1024 * 1024:  # 16 MB limit
    flash('File size exceeds the 16 MB limit. Please upload a smaller file.', 'danger')

    log = auditlog(
        user_id=session['user_id'],
        action_type='upload',
        details='Failed file upload - file too large: ' + original_filename,
        status='failed'
    )
    db.session.add(log)
    db.session.commit()
    return render_template('uploadfile.html', shared_with=shared_with,
                           description=description)

else:
```

Figure 21 File Uploading Function: File Checking and Logging for failed upload

Else, we add the file to the file model and to the database. We also add a new auditlog to signify the successful upload of the file.

```python
else:
    new_file = File(
        user_id=user_id,
        filename=unique_filename,
        original_filename=original_filename,
        filetype=filetype,
        file_size=file_size,  # Placeholder for file size
        file_data=file_data,
        description=description,
        shared_with=shared_with,
        status='pending',
        sensitivity=5,
        action='no action'
    )
    db.session.add(new_file)
    db.session.commit()

    # Logging the registration event
    log = auditlog(
        user_id=session['user_id'],
        action_type='upload',
        details='Successful file upload: ' + original_filename
    )
    db.session.add(log)
    db.session.commit()

    print(f"File {original_filename} uploaded by user {user_id}")
    flash(f'File "{original_filename}" uploaded successfully!', 'success')
    return redirect(url_for('main.view_files'))
```

Figure 22 File Uploading Function: Adding file to model and database + Logging event

Exception Handling is also done like in the register phase.

5.5 Logging

Logging is done by any action of registering, logging in, logging out, and uploading a file for now.

```
def auditlog(user_id, action_type, details='', status='success'):
    """Helper function to log audit events"""
    try:
        log_entry = AuditLog(
            user_id=user_id,
            action_type=action_type,
            details=details,
            status=status,
            timestamp=datetime.utcnow()
        )
        db.session.add(log_entry)
        return log_entry
    except Exception as e:
        print(f"Failed to log audit event: {e}")
        db.session.rollback()
```

Figure 23 Logging Function

The auditlog() function takes in parameters of the user id, action type, some small details of the action, status (success/failed) and the time of action.

```
    if user.is_admin:
        # Logging the login event for admin
        log = auditlog(
            user_id=session['user_id'],
            action_type='admin login',
            details='Successful login attempt'
        )
        db.session.add(log)
        db.session.commit()

        return redirect('/admin_dashboard')  # Or your admin route

    else:
        # Logging the login event for regular user
        log = auditlog(
            user_id=session['user_id'],
            action_type='login',
            details='Successful login attempt'
        )
        db.session.add(log)
        db.session.commit()
        return redirect(url_for('main.index'))  # Regular user dashboard
```

Figure 24 Logging function used on login event

This is the program for logging a user's activity of logging into the application.

We retrieve the user id and the action type, which could be success/failed, along with some small details, then add the log to the database before committing it to the application.

Server: localhost » Database: DLPS » Table: AuditLog

```
SELECT * FROM `AuditLog`
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None

Extra options

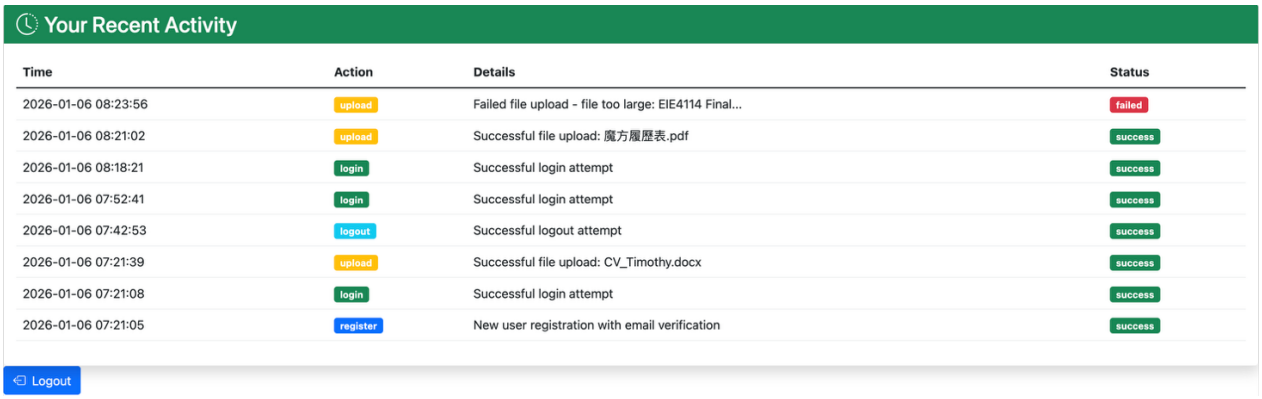
	log_id	user_id	action_type	details	status	timestamp
Edit Copy Delete	1	1	admin_create	Admin user created during database initialization	success	2026-01-06 15:20:23
Edit Copy Delete	2	1	admin login	Successful login attempt	success	2026-01-06 07:20:41
Edit Copy Delete	3	2	register	New user registration with email verification	success	2026-01-06 07:21:05
Edit Copy Delete	4	2	login	Successful login attempt	success	2026-01-06 07:21:08
Edit Copy Delete	5	2	upload	Successful file upload: CV_Timothy.docx	success	2026-01-06 07:21:39
Edit Copy Delete	6	2	logout	Successful logout attempt	success	2026-01-06 07:42:53
Edit Copy Delete	7	2	login	Successful login attempt	success	2026-01-06 07:52:41
Edit Copy Delete	8	2	login	Successful login attempt	success	2026-01-06 08:18:21
Edit Copy Delete	9	2	upload	Successful file upload: 魔方履歷表.pdf	success	2026-01-06 08:21:02
Edit Copy Delete	10	2	upload	Failed file upload - file too large: EIE4114 Final...	failed	2026-01-06 08:23:56

Check all *With selected:* Edit Copy Delete Export

Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None

Figure 25 AuditLog Table viewed on localhost/phpMyAdmin

The AuditLog table in the database would contain all logs from all users including the admin, to facilitate the viewing of logs by an admin, which is to be implemented in the future.



Your Recent Activity

Time	Action	Details	Status
2026-01-06 08:23:56	upload	Failed file upload - file too large: EIE4114 Final...	failed
2026-01-06 08:21:02	upload	Successful file upload: 魔方履歷表.pdf	success
2026-01-06 08:18:21	login	Successful login attempt	success
2026-01-06 07:52:41	login	Successful login attempt	success
2026-01-06 07:42:53	logout	Successful logout attempt	success
2026-01-06 07:21:39	upload	Successful file upload: CV_Timothy.docx	success
2026-01-06 07:21:08	login	Successful login attempt	success
2026-01-06 07:21:05	register	New user registration with email verification	success

Logout

Figure 26 Audit logs viewed from the user dashboard

If there are some exceptions/errors encountered (e.g. The uploaded file being too large thus cannot be uploaded, the event will also be logged but the action would be shown as ‘failed’ for its status.

This is done in some parts of code for now (e.g. the file uploading section).

```
#Read file data
file_data = file.read()
file_size = len(file_data)

if file_size > 16 * 1024 * 1024:  # 16 MB limit
    flash('File size exceeds the 16 MB limit. Please upload a smaller file.', 'danger')

    log = auditlog(
        user_id=session['user_id'],
        action_type='upload',
        details='Failed file upload - file too large: ' + original_filename,
        status='failed'
    )
    db.session.add(log)
    db.session.commit()
    return render_template('uploadfile.html', shared_with=shared_with,
                           description=description)
```

Figure 27 An example of an Audit log with status = ‘failed’

The above code means that if the file size is larger than 16MB, it won’t be uploaded to the database and will return a log with status “failed”.

6. Security Analysis

Some Cases of attacks in a normal file sharing system could be:

- Admin Leaking documents
- Users trying to attack the website by a Denial-of-service (DoS) Attack
- Storage Flooding/Database overload
- Exposure of data through screenshots of the data on the system, or hardware capture

These things can be prevented as follow.

Regarding admins, if we only give them the privileges of viewing all audit logs and changing user information upon request, documents wont be accessible by the admins and thus confidential documents wont be leaked by admins.

Regarding DoS Attacks and database overload, as we have users and admins who can see Audit Logs, we can trace back the source of leading to the attack or reason of database overload, such as a user uploading too many large files at once, and thus mitigate from there.

Regarding the exposure of data and capturing of confidential information, we can trace back the source of capture from the watermarks and the logging events on the system to see who the culprit is behind. Hardware capture of data is not avoidable, but we can do our best to trace the source and prevent it from happening from the same person again.

7. Elements of the project to be completed

The templates to be completed in the system are:

1. View_all_logs.html (admin dashboard)
2. Edit_file.html (Edit file)
3. View_file.html (View File)
4. Bin.html (Recycle Bin)

The elements to be completed regarding the files in the system are:

1. File Sharing function
2. File Viewing/editing function

3. File Encryption/Decryption
4. Watermark Processing function
5. File destruction: overwriting function
6. Recycle Bin Table in Database

The elements to be completed regarding AI models used are:

1. Risk Scoring AI Model: Used to flag a sensitivity value of sharing a specific document
2. AI model for flagging suspicious activity of login, logout, viewing/editing/sharing a file

The elements to be completed regarding other parts of the system are:

1. Restructure of Registration process: Should not allow user to register directly on the system, but instead should have an email sending them a one-time link that will bring them to the register page
2. OTP function: Used for email verification
3. Admin Logs: Admin being able to view all logs
4. Account Settings: Admin being able to change user's account information upon their request

The elements to be completed regarding the deploying of the system on the internet are:

1. CSRF Tokens
2. SSF Certificate for HTTPS
3. Domain name

8. Conclusion

Our data, our information has slowly been changed into digital form. Although this makes our lives more convenient and faster, it also made our data and our privacy slowly became more vulnerable, and eventually may even disappear.

Data security is a rising problem, and the world knows it. As hackers start to emerge, large companies start to become victims of data breaches, storing data in digital forms start to become a risk. As hacking methods only will increase over time, there isn't an "always safe" option for us to keep our data secure forever. However, we can try our best to mitigate the risks and implement new methods to keep our information secure. Although it is a challenging task, it is still possible.

Many companies have their own DLP policies and methods to achieve DLP, but still fell to hacking and data breaching incidents, and for my Data Leakage Prevention System, I hope to prevent these things from happening to the users of this system and bring a safer browsing experience for them, by keeping their data secure.

References

[1] "Yahoo 2013 data breach hit 'all three billion accounts'", bbc.com, Oct. 4, 2017. [Online]. Available: https://www.bbc.com/news/business-41493494 [Accessed Oct. 20, 2025].

[2] N. Confessore, "Cambridge Analytica and Facebook: The Scandal and the Fallout So Far", The New York Times, Apr. 4, 2018. [Online]. Available: https://www.nytimes.com/2018/04/04/us/politics/cambridge-analytica-scandal-fallout.html [Accessed Oct. 20, 2025]

[3] J. Greig, "PayPal penalized $2 million over data breach involving 35K Social Security numbers", The Record, Jan. 25, 2025. [Online]. Available: https://therecord.media/paypal-penalty-millions-data-breach [Accessed Oct 21, 2025]

[4] Digital Policy Office, Data Breach, 2025. [Online]. Available: https://www.infosec.gov.hk/en/knowledge-centre/data-breach [Accessed Oct. 20, 2025]

[5] Microsoft Pureview Information Protection (MPIP), Spirion Sensitive Data Governance Suite, Aug. 15, 2023 [Online]. Available: https://www.spirion.com/marketplace/microsoft-purview-info-protection [Accessed Jan. 03, 2026]

[6] Advanced Encryption Standard (AES), GeeksForGeeks, Aug. 08, 2025 [Online]. Available: https://www.geeksforgeeks.org/computer-networks/advanced-encryption-standard-aes/ [Accessed Jan. 04, 2026]

[7] DoD 5220.22-M Explained – Data Erasure Standards, Jetico, Mar. 21, 2023 [Online]. Available: https://jetico.com/blog/dod-522022-m-explained-data-erasure-standards/ [Accessed Jan. 04, 2026]

[8] Python Pillow – Creating a Watermark, GeeksForGeeks, Jul. 23, 2025 [Online]. Available: https://www.geeksforgeeks.org/python/python-pillow-creating-a-watermark/ [Accessed Jan. 04, 2026]

[9] PyMuPDF documentation, Dec. 04, 2025 [Online]. Available: https://pymupdf.readthedocs.io/en/latest/ [Accessed Jan. 09, 2026.]

[10] L. Matthews, “A Nasty Safari Bug Could Leak Your Browsing Data And Google Account Information”, Forbes, Jan. 17, 2022 [Online]. Available: https://www.forbes.com/sites/leemathews/2022/01/17/a-nasty-safari-bug-could-leak-your-browsing-data-and-google-account-information/ [Accessed Jan. 03, 2026]

[11] Trautman, Lawrence J. and Ormerod, Peter C, "Corporate Directors' and Officers' Cybersecurity Standard of Care: The Yahoo Data Breach," American University Law Review: Vol. 66: Iss. 5, Article 3, 2017.